0.1 Introduction

Chapter 3 produced a validated graph object that carries the network topology together with its layered iteration sets and ancestry closures, and every analysis in the framework reads that object as it is. This chapter presents the Network Decomposition Module, a second pre-processing step which, when invoked, extends the graph object with the network's decomposition structure. Its objective is to identify every instance of a specified subgraph pattern in the input DAG. The default pattern is the diamond, the fork-and-join structure that Chapter 3 identified as the redundancy of engineered systems. In the present work the module is consumed by the Probability Propagation Toolkit of Chapter 5, which conditions on diamonds to compute exact reachability. The Critical Path Toolkit of Chapter 7 does not use it. Since each identified subgraph is returned as a graph object of the same form as the network, the module is available to any further analysis that a user adds.

Section 0.1.1 places the module among the decomposition strategies used in network analysis. Section 0.2 describes the three discriminators that define a pattern, and Section 0.3 defines the diamond in their terms. Section 0.4 presents the identification algorithm and proves its grouping, termination, edge-isolation and identity properties. Section 0.5 proves global completeness and determinism. Section 0.6 describes the unique diamond object, and Section 0.7 traces the algorithm on three small networks.

0.1.1 Decomposition in Complex Networks

Real infrastructure networks are rarely trees. Their components connect to and depend on many others at once, and the interactions between them overlap instead of branching cleanly [1, 2]. Redundancy is one cause. A robust network supplies more than one route to the same goal, so paths diverge and reconverge. Process networks and hierarchical flows behave the same way, with a component feeding several downstream dependencies while itself depending on shared upstream outputs. As the system grows, overlapping paths accumulate.

Graph-based analysis methods almost always do some form of decomposition, and for large networks with complicated connectivity it is a practical necessity [3]. Broadly, decomposition breaks a network into substructures that are easier to handle than the whole, and methods differ in what they do with those substructures. Some strategies aim to reduce: a substructure is replaced by a single equivalent element, so the network shrinks while the analysis stays exact. Other decomposition methods work by partition: the network is divided into units of work sized for the solver, and the pieces are processed in turn. Beyond either, decomposition can give a more compact representation of the network, storing a repeated substructure once instead of every time it appears, without losing the overall structure. Depending on how the decomposed units are defined, they

can also support localised investigation, such as confining a sensitivity study to one part of the system without needing to run analysis against the full network.

There is no universal correct unit of decomposition. A decomposition method defines its own target substructure and partitions the network in those terms. In the series–parallel reduction reviewed in Chapter 2 the units are the series and parallel pairs themselves [4]. In factoring the unit is a single component, on which the method conditions to split the problem into the working and failed cases [5]. In tree decomposition the unit is a bag of nodes, and a tree of overlapping bags is the subproblem structure a dynamic program can process in a single sweep [6, 7].

The framework's default decomposition unit is the *diamond*, a fork–join subgraph in which paths fan out from a common node and fan back in at a node further downstream. The decomposition is partition-forward, that is, each identified diamond is returned as a localised graph object of the same form as the full network and can be handed to any analysis module in the framework on its own. Secondarily, a diamond can serve as a reductive unit, since a solved diamond can stand in for its subgraph as a single equivalent component. This is how the Probability Propagation Toolkit uses it.

0.2 A Generic Decomposition Module

The network decomposition module identifies and returns every instance of a subgraph pattern discovered in the input DAG. The specific subgraph pattern is decided by three discriminator functions:

- an *eligibility* discriminator, which decides which nodes may take part in a pattern at all;
- an *anchoring* discriminator, which decides what structure a pattern forms around;
- a *membership* discriminator, which decides when two paths into a node belong to the same pattern.

The identification algorithm does the same work whatever the discriminators say.

A discriminator can be defined on any property of the graph object, or on any network characteristic mapped to a property of it during input processing, and each discriminator can be overridden or modified to fit the required definition. For example, the membership threshold can be raised from two paths to three, or the anchoring structure can be changed from fan-out-then-fan-in to fan-in-then-fan-out. A discriminator can also be defined on a characteristic that is not already a graph object property, provided it is mapped onto one during input processing. A component type supplied with the raw network, for instance, can be mapped to a node property so that only one class of component can anchor a pattern.

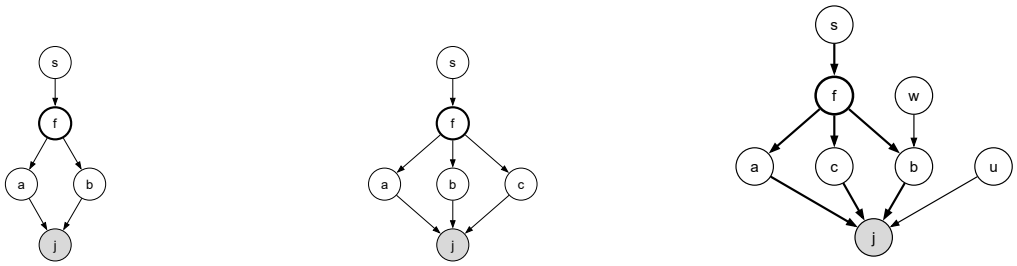
In the Julia implementation of the framework, the eligibility discriminator is exposed as a runtime override, and the other two are small internal functions that a replacement definition would replace.

While the module stands as a generic subgraph decomposition tool, the discussion in the rest of this chapter mainly focuses on the framework's default target, the *diamond* subgraph.

0.3 The Diamond Subgraph

As introduced in Section 0.1.1, the decomposition unit in this work is the diamond subgraph. Diamond patterns are formed when paths diverge from a common fork and reconverge downstream with overlapping ancestries.

Consider, for example, a distribution system where two supply routes leave one substation and meet again at a demand point. A diamond is this diverge-and-reconverge pattern. An *induced* diamond is a diamond made of one fork and one diamond-forming join. Every one of its paths originates from that single fork, whether there are two of them or four, and the subgraph carries no second diamond join (Figure 1). However, diamonds are not always induced. Non-diamond paths may still reach the same join node, as in Figure 1c. As network interconnectivity increases, so does the likelihood of diamond patterns and their complexity, with diamonds becoming more nested and overlapping each other's paths (Figure 2).

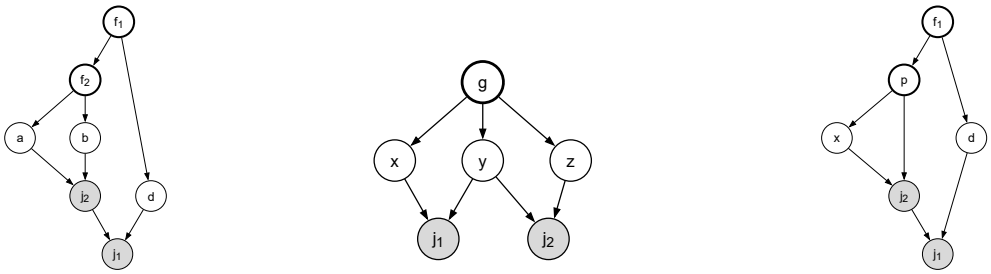


(a) Two paths. (b) Three paths. (c) With external edges.

Figure 1: Induced diamonds. Every diamond path originates from the single fork and the subgraph carries one diamond join.

The decomposition module consumes the DAG $G = (V, E)$ and the iteration sets $L = \{L_1, \ldots, L_k\}$ from the graph object of Chapter 3. Given the iteration sets, the topological index $\text{topi} : V \to \{1, \ldots, |V|\}$ follows by numbering nodes in increasing order across layers and in sorted node order within each layer, so every node receives a distinct integer and topi is a strict total order on V refining the layered order.

The diamond in the framework is thus defined by the three discriminators.



(a) Nested. (b) Overlapping. (c) Self-influencing, nested.

Figure 2: Non-induced diamonds. In (b) two diamonds overlap through the shared fork g, and in (c) parent p is itself the fork of the inner pattern.

- Eligibility criterion: the node's declared value must leave its reachability unsettled.
- Anchoring structure: the shared fork from which the reconvergent paths originate.
- Membership criterion: two parents belong to the same pattern when they share ancestry.

The eligibility criterion is the one discriminator not drawn from the diamond's definition, and `is_det` in the implementation is the only place identification reads declared values. Since the module was built with probabilistic propagation in mind, the default eligibility criterion excludes nodes whose prior probability is deterministic. This covers two cases: any inactive node (i.e. unreachable with prior = 0) or a perfect source node (i.e. always active with prior = 1). The reason for the conjunction in the second case is that a non-source node with prior = 1 may still be probabilistically uncertain, depending on its incoming edges. Therefore excluding every node valued 0 or 1 would incorrectly disable every diamond in a probabilistic DAG where all node priors happen to be 1. For interval-valued and p-box-valued networks the two cases apply only to degenerate values, an interval or p-box collapsed exactly onto 0 or 1.

Definition 1 (Diamond join)**.** *A join node is a* diamond join *when an unsettled fork lies in the shared ancestry of two of its parents, or one unsettled parent is itself an ancestor of another, so that a diamond forms there. A diamond join can itself be a fork of a different diamond elsewhere in the network.*

Each parent arriving at a diamond join carries its *influencing set*, its ancestry under the self-inclusive closure, the nodes with a path to it and the parent itself. A diamond join can also receive paths that originate outside the fan-out, and these form its non-influencing set.

Definition 2 (Non-influencing set)**.** *The* non-influencing set *of a diamond join is the set of its parents whose influencing sets meet no other parent's. Their edges enter the*

join from outside every reconvergent pattern there and are carried alongside the join's diamonds as independent contributions.

Given this, the diamond subgraph is defined formally.

Definition 3 (Diamond)**.** *A diamond D is a subgraph of G whose relevant node set $R \subseteq V$ spans a diamond join, its correlated parents, and everything upstream of them, together with a set $C \subseteq V$ of fixed nodes that isolate the pattern.*

The framework exposes two views of the network's decomposed subgraphs.

Unique diamonds are the complete set of distinct diamonds found in the network, each stored once, in `unique_subgraphs`. Any node can participate in any number of unique diamonds, whether as a fan-out point, a fan-in point, or an intermediate node along the paths.

The maximal diamond is the network-level view of a diamond join. It is the full subgraph the join's reconvergence forms, made up of the diamond of every group of the join's parents that forms one, spanning the join, the groups, and everything upstream of them, together with the edges of the join's non-influencing set. Therefore every diamond join has at most one maximal diamond, while a fork can participate in several. Every diamond found at a deeper level inside a maximal diamond is a **sub-diamond**, and its edge list is contained in the enclosing diamond's.

Two consequences follow from these definitions. A unique diamond with no inner sub-diamond has one fork and one diamond join, so it is an induced diamond, the smallest a diamond can be. Furthermore, since every diamond is either maximal at its join or a sub-diamond of a maximal one, the set of unique diamonds is the set of distinct maximal diamonds together with their inner sub-diamonds.

0.4 The Identification Algorithm

`identify_diamonds` is a recursive set-membership algorithm over the closures of the graph object. For every join node of G it identifies the join's maximal diamond, pairing the join with the earliest covering forks in its parents' shared ancestry, and it recurses inside each diamond it forms for the sub-diamonds. Each call works on a join v, a *fixed context* $E \subseteq V$ of nodes already fixed by enclosing diamonds, and the parents P of v in view. The recursion starts from $E = \varnothing$, and Algorithm 1 shows the skeleton.

Under a context each parent's influencing set shrinks to

$$\mathrm{infl}(p, E) \;=\; \mathrm{Anc}(p) \setminus E,$$

with $\mathrm{Anc}(p)$ self-inclusive so that $p \in \mathrm{infl}(p, E)$ whenever $p \notin E$. Two parents p, q of a join are *structurally correlated under* E when $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E) \neq \varnothing$, that is, when

some node upstream of both is not held fixed. Similar to factoring based decomposition, extending the context with a node removes it from every influencing set.

Algorithm 1 Recursive diamond identification (skeleton)

Require: join node v, fixed context E, parents P of v in view
Ensure: the diamonds at v under E

```
partition P into groups by pairwise influencing-set intersection
for all groups G with |G| ≥ 2 do
    f ← eligible covering fork of G with minimum topi
    if no eligible covering fork exists then
        members of G join the non-influencing set of v; skip the group
    end if
    build diamond D for (v, G, f) under B = E ∪ {f}
    reuse the stored entry if the identity key of D is already in unique_subgraphs
    for all join nodes j within D do
        recurse on (j, B, parents of j), restricted to G when j = v
    end for
end for
singleton groups join the non-influencing set of v
```

0.4.1 Shared ancestry and the earliest covering fork

At a join v with parents P under context E, any two parents whose influencing sets intersect are merged into one group by union–find.

Theorem 4 (Grouping correctness)**.** *Fix a join v, context E, and parent set P. Let H be the undirected graph on vertex set P with an edge between p and q if and only if* $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E) \neq \varnothing$. *Then* `components`$(P, E)$ *returns exactly the connected components of H.*

Proof. The intersection $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E)$ is evaluated for every unordered pair of P, so the pairs that intersect are exactly the edges of H, and union–find merges exactly those pairs. Union–find over a vertex set and its edge relation returns the connected components, which for P under this edge relation is the component partition of H. □

Theorem 5 (Group disjointness)**.** *If $G_i \neq G_j$ are distinct groups returned by* `components`(P, E)*, then*

$$\Big(\bigcup_{p \in G_i} \mathrm{infl}(p, E)\Big) \cap \Big(\bigcup_{q \in G_j} \mathrm{infl}(q, E)\Big) = \varnothing.$$

Proof. Let $p \in G_i$ and $q \in G_j$. By Theorem 4 they lie in different components of H, and since every pair was evaluated, $\mathrm{infl}(p, E) \cap \mathrm{infl}(q, E) = \varnothing$. The intersection of the two unions expands to the union of these pairwise intersections, each of which is empty. □

Disjointness here is a statement about the influencing sets of the diamond groups, namely that each group decomposes separately. Whether separately computed values

recombine correctly at the join is a different claim, which Chapter 5 makes and proves for the probabilistic reading. Parents in singleton groups fall into the join's non-influencing set.

Each group of two or more is then searched for its eligible covering forks, those lying in the shared ancestry of at least two members and not already in the fixed context, with a member that is an ancestor of another counting as its own cover. The diamond anchors on the earliest of them, the covering fork of minimum topological index, which leaves the largest part of the group's shared structure intact downstream for reuse, and the strict total order makes the choice unique. A group whose shared ancestry offers no eligible covering fork forms no diamond, and its members join the non-influencing set.

0.4.2 Recursive flow

Theorem 6 (Termination)**.** *The recursion of Algorithm 1 terminates after at most $|V|$ nested context extensions along any recursive path.*

Proof. Along any path of the recursion, each call either returns without recursing, which happens when every group at the join in view is a singleton or offers no covering fork, or fixes a fork f and recurses with $B = E \cup \{f\}$. Covering forks are never already in E (Section 0.4.1), so $f \notin E$ and $|V \setminus B| = |V \setminus E| - 1$ strictly. The quantity $|V \setminus E|$ is a non-negative integer that strictly decreases along the recursive path, and from its initial value $|V \setminus \varnothing| = |V|$ it admits at most $|V|$ decrements. □

Fixing fork f for group G at join v yields the diamond with boundary $B = E \cup \{f\}$. Paths within the diamond can share further unfixed ancestry, so the recursion runs again at every join within it, the diamond join itself included, under the extended context. Sub-diamonds form inside maximal diamonds this way until no covering fork and join pair remains anywhere.

0.4.3 Subgraph construction

The diamond for group G at join v under boundary B spans

$$R \;=\; \{v\} \cup G \cup \bigcup_{p \in G} \mathrm{Anc}(p),$$

the join, the group, and everything upstream of the group, with induced edge list

$$\mathcal{E}_D \;=\; \big\{\, (u, w) \in E \;:\; u, w \in R,\;\; w \notin B,\;\; (w \neq v \text{ or } u \in G) \,\big\}.$$

Write $V(\mathcal{E}_D)$ for the set of its endpoints. The first two clauses keep the edge list on the relevant set R. The third removes every edge into a fixed node, whose upstream structure

is already settled, while its outgoing edges stay and act as local sources of the diamond. The fourth admits an edge into the diamond join only from the group. The reason for the fourth clause is given after the lemma.

Lemma 7 (Edge isolation)**.** *Let D be constructed for group G at join v under boundary B as above. Then $(u, w) \in \mathcal{E}_D$ if and only if $(u, w) \in E$, $u, w \in R$, $w \notin B$, and ($w \neq v$ or $u \in G$). Consequently every directed path within $\mathcal{E}_D$ that terminates at v has its final edge originating in G, and no node of B appears on any path within $\mathcal{E}_D$ except as its first node.*

Proof. The membership characterisation restates the filter defining $\mathcal{E}_D$. For the path properties, induct on path length. A length-one path into v is a single edge $(u, v) \in \mathcal{E}_D$, which the fourth clause forces to have $u \in G$. Any longer path has a final edge satisfying the same property, and each preceding edge (u, w) has $w \notin B$ and both endpoints in R by the first three clauses, so no node after the first lies in B and no edge of the path enters from outside R. □

Consider a nested level at which an enclosing diamond has already fixed a fork r, so $r \in E$, and suppose r is both an ancestor of some group member p and a direct parent of the join v. Partitioning places r correctly, since fixing removed r from every influencing set, and in the minimal case of r a source with no free ancestry its influencing set is empty, landing it in a singleton group whose contribution to v enters separately.

However, the span R is built from the raw closure $\mathrm{Anc}(p)$ and contains r whatever the context says, so the partition and the span disagree wherever $E \neq \varnothing$. The edge (r, v) then has both endpoints in R and a head outside B, so the first three clauses admit it, and a diamond meant to capture v as reached through G alone gains a second, unrelated entry into its own join. The fourth clause blocks this edge. Figure 3 shows the minimal instance.

0.4.4 Context-aware identity

A diamond's edge list alone cannot place it in the nesting hierarchy.

Proposition 8. *(i) There exist graphs and contexts $E_A = \varnothing$ and $E_B = \{g\}$, with g a source fork among the endpoints of $\mathcal{E}_D$, under which the construction of Section 0.4.3 anchors the same fork f and yields the same edge list $\mathcal{E}_D$, while one diamond is maximal and the other a sub-diamond. The pair $(\mathcal{E}_D, \{f\})$ therefore conflates distinct diamonds. (ii) Under* $\mathrm{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap V(\mathcal{E}_D)))$*, equal keys imply the same diamond join, the same group, the same edge list, and the same set C, with the two fixed contexts agreeing on every node of $V(\mathcal{E}_D)$ and every stored field equal.*

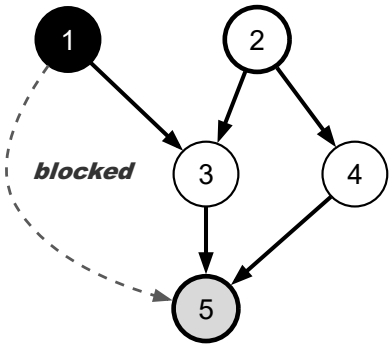


Figure 3: Edge isolation at join 5 in context $E = \{1\}$. Node 1 (filled) was fixed by an enclosing diamond and is excluded from the group $G = \{3, 4\}$ with fork 2. Its direct edge $(1, 5)$, drawn dashed, is admitted by the first three filter clauses but blocked by the fourth. Bold edges form the nested diamond's edge list.

Proof. For (i), take a join v whose group G anchors on fork f, and a source fork g that is an ancestor of exactly one member of G and keeps one edge whose head lies outside both boundaries. Coverage of one excludes g from the covering-fork search, so the search offers the same forks in both cases and f is anchored in both, and removing g from a single influencing set cannot split the group, since the correlation runs through f. The cut into g removes nothing, as a source has no incoming edges, and g's retained edge passes the filter in both cases, so $g \in V(\mathcal{E}_D)$. The group, the fork, the span, and the edge list therefore coincide, while the diamond is maximal under E_A and a sub-diamond of the one that fixed g under E_B. The keys differ, since $E_A \cap V(\mathcal{E}_D) = \varnothing$ and $g \in E_B \cap V(\mathcal{E}_D)$.

For (ii), equal keys give equal $\mathcal{E}_D$ and equal C, since both are hashed. The diamond join is recoverable as the unique endpoint of $\mathcal{E}_D$ that every other endpoint precedes in the closure, which acyclicity makes unique, and the group as the in-neighbourhood of the join in $\mathcal{E}_D$, which the fourth clause admits from the group alone. Every stored field is computed from $\mathcal{E}_D$, $V(\mathcal{E}_D)$, C, and the global closures, so equal inputs give equal fields, and $B \cap V(\mathcal{E}_D) = (\{f\} \cap V(\mathcal{E}_D)) \cup (E \cap V(\mathcal{E}_D)) = C \cap V(\mathcal{E}_D)$ in both constructions. The two fixed contexts can still disagree away from the diamond, but only narrowly. A node outside the span is never consulted, since every membership test in the recursion concerns a parent of a join within the diamond or one of its ancestors, all inside the span by transitivity. A span node x in one fixed context and not the other has no incoming edge, since such an edge would have its tail in the span and would pass the filter where x is free, forcing $x \in V(\mathcal{E}_D)$, and each outgoing edge of x inside the span is absent from the shared $\mathcal{E}_D$, so its head lies in each construction's boundary or it is a direct edge into the join excluded by the fourth clause. Every path from such a node towards the diamond therefore begins at a node in both fixed contexts or at an excluded edge. □

Since a diamond cannot be unique by its edge list and fork alone, every unique diamond is keyed by

$$\mathrm{hkey}(D) \;=\; \big(\mathcal{E}_D,\ \{f\} \cup (E \cap V(\mathcal{E}_D))\big),$$

where $\mathcal{E}_D$ is the diamond's edge list, f its covering fork, and $E \cap V(\mathcal{E}_D)$ the fixed context restricted to the diamond's endpoints.

Before a new entry is built, the key is looked up in `unique_subgraphs`, and a structurally identical diamond already resolved anywhere in the recursion is referenced instead of rebuilt. This is hash-consing [8, 9], a table keyed by structural identity so that equal structures are stored once and identity comparison becomes a key comparison, sometimes used in BDD algorithms. Without the store the recursion is a tree and every call site is distinct. With it, structurally identical call sites collapse into shared entries and the representation becomes a DAG. Diamonds with the same identity share one stored entry regardless of the outer context, since Proposition 8 bounds whatever influence a node outside the identity carries towards the diamond, and the shared entry meets the same fixed context within.

0.5 Structural Correctness

0.5.1 Global completeness

Theorem 9 (Global completeness)**.** *Let v be a join node of G with parent set P, and suppose there exist distinct $p, q \in P$ and an unsettled node a such that either a is a fork node in $\mathrm{Anc}(p) \cap \mathrm{Anc}(q)$, or $p \in \mathrm{Anc}(q)$ (the asymmetric case, taking $a = p$). Then the top-level pass of `identify_diamonds` stores a maximal diamond for v. Conversely, a join admitting no such pair receives none.*

Proof. At the top level $E = \varnothing$, so in either case $a \in \mathrm{infl}(p, \varnothing) \cap \mathrm{infl}(q, \varnothing)$ and the intersection is non-empty. By Theorem 4, p and q lie in a common group G with $|G| \geq 2$. In the covering-fork search, a counts once for each member it covers. In the symmetric case a is a fork in both ancestries and covers p and q. In the asymmetric case $a = p$ is necessarily a fork, since it has an edge to v and a path to q that cannot pass through v, and with the self-inclusive closure it covers itself as well as q. Either way the coverage reaches 2 and the set of covering forks is non-empty. A non-empty finite set has a topi-minimal element, so the search anchors some fork, not necessarily a itself, Algorithm 1 constructs a diamond for G, and the top-level pass records it under v. For the converse, suppose no such pair exists. Any covering fork the search admits would yield one, since a symmetric cover is a fork in two members' ancestries, an asymmetric cover is a parent in another member's ancestry, and the search admits only unsettled nodes. Every group therefore offers no covering fork, whether its members are unrelated or related only through settled

structure, and the algorithm constructs nothing at v. That is the correct outcome, because whatever shared ancestry remains at v is already settled. □

0.5.2 Determinism

Theorem 10 (Determinism)**.** *For a fixed input graph and iteration sets (G, L), join v, context E, and admissible parent set, the diamonds returned by Algorithm 1 are uniquely determined, independent of set or dictionary iteration order, thread scheduling, or any other accident of execution.*

Proof. By induction on recursion depth, it suffices that each step of one call is a function of its arguments alone. This holds for the partition step because, by Theorem 4, its output is the connected-component partition of H, a graph invariant that does not depend on the order in which pairs are examined or merges performed. It holds for the selection step because the set of covering forks is itself a function of the arguments, and either it is empty, in which case the group deterministically produces no diamond, or the diamond anchors on its minimum under topi, which a strict total order makes unique. The boundary $B = E \cup \{f\}$ and span R are then determined, hence so is $\mathcal{E}_D$, and hence so are the joins passed to the recursive calls, which are deterministic by the inductive hypothesis. The store adds one further mechanism, reuse of an earlier entry on a key hit. Proposition 8(ii) makes equal keys carry the same join, group, edge list, and stored fields, and confines any residual difference between merged call sites to nodes outside the identity, so reuse introduces no dependence on visit order. □

This guarantee is similar to the canonicity guarantee of reduced ordered binary decision diagrams [10]. While BDD canonicity depends on an externally supplied variable ordering, topi is derived from a property of the input graph.

0.6 The Unique Diamond Object

The algorithm builds each unique diamond as a graph object of its own. The unique diamond object carries the diamond's characteristics together with its subgraph in the same form as the network's graph object, so a diamond can be treated as a standalone DAG network for any other type of analysis in the framework. Table 1 lists its fields.

Invoking the decomposition module on the graph object of Chapter 3 extends the object with the field `unique_subgraphs`, holding the network's unique subgraphs. The Probability Propagation Algorithm of Chapter 5 reduces each solved diamond to a supernode. The concrete Julia types behind the fields of Table 1 are described in Chapter 8.

Table 1: Fields of the unique diamond object

Field	Role
`sub_outgoing_index`	Forward adjacency of the diamond
`sub_incoming_index`	Reverse adjacency of the diamond
`sub_sources`	Local sources
`sub_fork_nodes`	Local forks
`sub_join_nodes`	Local joins
`sub_ancestors`	Ancestor closure restricted to the diamond's nodes
`sub_descendants`	Descendant closure restricted to the diamond's nodes
`sub_iteration_sets`	Layered order restricted to the diamond's nodes
`sub_node_priors`	Value map restricted to the diamond's nodes
`sub_diamond_structures`	For each join within this diamond, its sub-diamonds, held by their keys into `unique_subgraphs`, and its non-influencing set
`is_maximal`	Whether the diamond is maximal
`diamond`	The span R, the set C, and the edge list $\mathcal{E}_D$

0.7 Worked Examples

0.7.1 The induced diamond

Figure 1a is the minimal pattern, a source s feeding a fork f, two internally disjoint paths, and a join j where they meet. At j both parents' influencing sets contain f, so the parents form one group. Fork f is the only covering fork and is fixed, and the diamond spans the whole figure with $C = \{f\}$.

0.7.2 A factorised join

The nine-edge network of Figure 4 has forks 1 and 2 and a single join at node 7, which source 8 also feeds directly. The parents split into two correlated groups, $\{3, 4\}$ through fork 1 and $\{5, 6\}$ through fork 2, with parent 8 related to neither. Two constituent diamonds arrive at join 7, one anchored on each fork, and node 8 is carried in the non-influencing set. Together they are the join's maximal diamond.

0.7.3 A nested decomposition

The eight-node network of Figure 5 has forks 1 and 3 and joins 6 and 8. An outer reconvergence runs through fork 1 and meets at join 8, and on one of its branches an inner reconvergence runs through fork 3 and meets at join 6. The decomposition below is the machine-checked output of the implementation on this edge list, and the examples of Figures 3 and 4 were verified in the same run.

Join 8, top level. The parents are $\{2, 7\}$, with $\mathrm{infl}(2, \varnothing) = \{1, 2\}$ and $\mathrm{infl}(7, \varnothing) =$

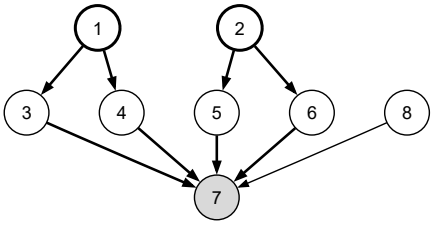


Figure 4: A factorised join. Two correlated groups meet at join 7, one through each fork, and source 8 enters from outside both. Bold edges mark the two constituent diamonds.

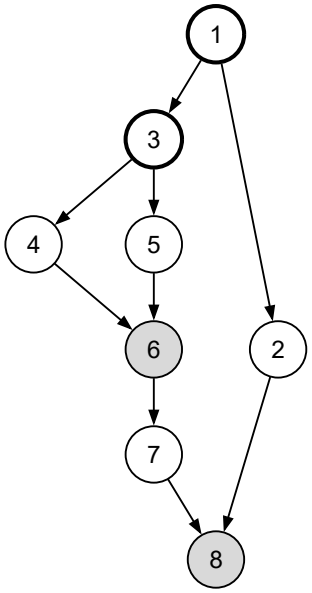


Figure 5: Frame 1. The network, with forks outlined in bold and joins shaded.

$\{1, 3, 4, 5, 6, 7\}$. The sets intersect in $\{1\}$, so the parents form one group. Fork 1 is the only covering fork of the pair and is fixed, and the maximal diamond D_1 spans all nine edges with $C = \{1\}$ (Figure 6).

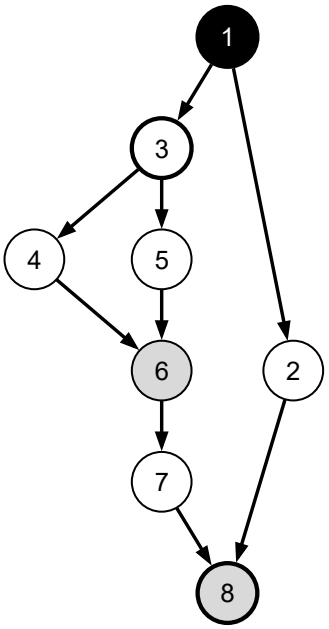


Figure 6: Frame 2. The maximal diamond at join 8, with fork 1 fixed (filled) and the span covering the whole network.

Join 6, inside D_1. Under the boundary $B = \{1\}$, join 6 has parents $\{4, 5\}$ with $\mathrm{infl}(4, \{1\}) = \{3, 4\}$ and $\mathrm{infl}(5, \{1\}) = \{3, 5\}$. The parents are still correlated through fork 3, so fixing 1 did not resolve the inner reconvergence. The recursive call fixes fork 3 and produces the inner diamond D_3 with $C = \{3\}$ and $\mathcal{E}_D = \{(3, 4), (3, 5), (4, 6), (5, 6)\}$. The edge $(1, 3)$ is cut by the boundary clause because its head is fixed at this level (Figure 7).

Join 6, top level. Join 6 is also a join of the network itself, so the top-level pass visits it in its own right. With $E = \varnothing$, forks 1 and 3 both cover parents 4 and 5, and the earliest covering fork rule fixes fork 1 and not fork 3. The result is a second maximal diamond D_2 spanning $\{(1, 3), (3, 4), (3, 5), (4, 6), (5, 6)\}$ with $C = \{1\}$, and the recursion inside it, now in context $\{1\}$, again produces exactly D_3.

The emitted hierarchy. In total the module returns two maximal diamonds and three unique diamonds (Figure 8). D_1 sits at join 8 and D_2 at join 6, each with $C = \{1\}$. The inner D_3 at join 6 with $C = \{3\}$ is stored once and referenced from the substructure tables of both D_1 and D_2 under the same key. Two different enclosing recursions posed the same sub-problem, with the same edge list, the same fork, and contexts agreeing on the span. Proposition 8(ii) is the reason the single shared entry is correct.

The trace also confirms the edge-isolation case of Section 0.4.3 end to end. In the

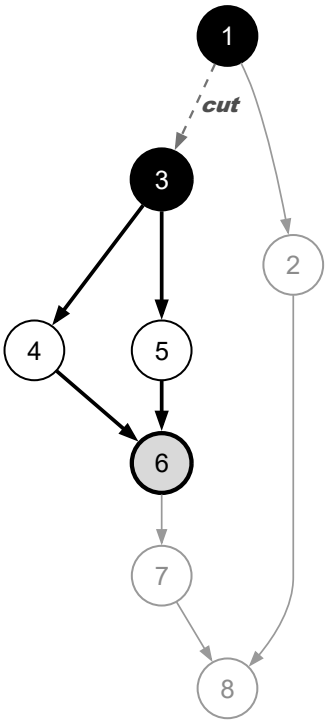


Figure 7: Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 fixed, the inner diamond in bold, and edge $(1, 3)$ cut.

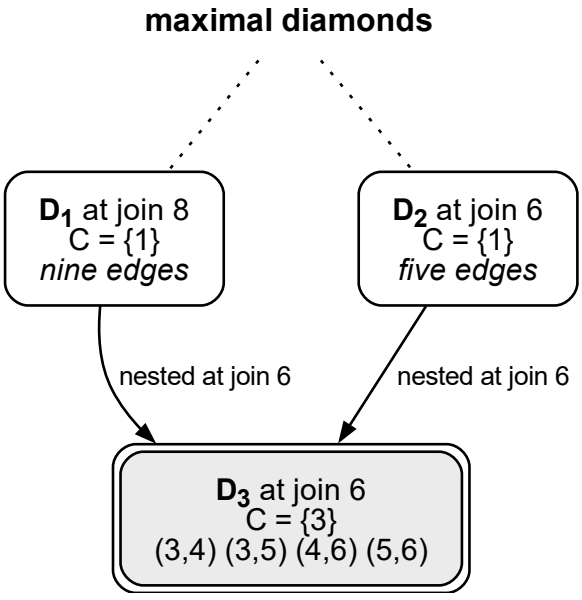


Figure 8: Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.

network of Figure 3, the nested diamond at join 5 under $E = \{1\}$ is emitted with $C = \{2\} \cup (\{1\} \cap V(\mathcal{E}_D)) = \{1, 2\}$, node 1 staying among the edge list's endpoints through its surviving edge $(1, 3)$, which is the context-aware identity of Section 0.4.4 visible in the output. Node 1 is recorded in the nested diamond's non-influencing set, and the edge $(1, 5)$ appears nowhere in its edge list. At the top level node 1 is a member of the group itself, so the maximal diamond at join 5 carries $(1, 5)$ legitimately.

0.8 Summary

The module extends the graph object of Chapter 3 with `unique_subgraphs`, storing every diamond found at any level exactly once, each entry a graph object of the same form as the network it came from, with the maximal diamonds flagged. The decomposition terminates within $|V|$ nested steps and is canonical for a given input (Theorems 6 and 10). It groups a join's parents exactly by shared unfixed ancestry and produces a diamond wherever unsettled reconvergence exists and only there (Theorems 4–9). Nothing outside a diamond's span enters its edge list (Lemma 7), and the context-aware identity of Section 0.4.4 distinguishes a maximal diamond from a sub-diamond with the same edge list and fork (Proposition 8).

Bibliography

[1] M. Ouyang, "Review on modeling and simulation of interdependent critical infrastructure systems," *Reliability Engineering & System Safety*, vol. 121, pp. 43–60, 2014. DOI: 10.1016/j.ress.2013.06.040

[2] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, and S. Havlin, "Catastrophic cascade of failures in interdependent networks," *Nature*, vol. 464, no. 7291, pp. 1025–1028, 2010. DOI: 10.1038/nature08932

[3] H. Pérez-Rosés, "Sixty years of network reliability," *Mathematics in Computer Science*, vol. 12, pp. 275–293, 2018. DOI: 10.1007/s11786-018-0345-5

[4] A. Satyanarayana and R. K. Wood, "A linear-time algorithm for computing K-terminal reliability in series-parallel networks," *SIAM Journal on Computing*, vol. 14, no. 4, pp. 818–832, 1985. DOI: 10.1137/0214057

[5] A. Satyanarayana and M. K. Chang, "Network reliability and the factoring theorem," *Networks*, vol. 13, no. 1, pp. 107–120, 1983. DOI: 10.1002/net.3230130107

[6] N. Robertson and P. D. Seymour, "Graph minors. II. Algorithmic aspects of tree-width," *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986. DOI: 10.1016/0196-6774(86)90023-4

[7] A. K. Goharshady and F. Mohammadi, "An efficient algorithm for computing network reliability in small treewidth," *Reliability Engineering & System Safety*, vol. 193, p. 106 665, 2020. DOI: 10.1016/j.ress.2019.106665

[8] A. P. Ershov, "On programming of arithmetic operations," *Communications of the ACM*, vol. 1, no. 8, pp. 3–6, 1958, The origin of the hash-consing technique. DOI: 10.1145/368892.368907

[9] J.-C. Filliâtre and S. Conchon, "Type-safe modular hash-consing," in *Proceedings of the ACM SIGPLAN Workshop on ML*, 2006, pp. 12–19. DOI: 10.1145/1159876.1159880

[10] R. E. Bryant, "Graph-based algorithms for Boolean function manipulation," *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986. DOI: 10.1109/TC.1986.1676819